

 pilk Public

Watch 0Fork 11Star 55

main 1 Branch 5 TagsGo to fileAdd fileCode

 lemisky	update doc	1855209 · 5 months ago	37 Commits
.github/workflows	update pilk	2 years ago	
pilk	修复 get_duration 结果不对问题	9 months ago	
src	fix PilkError multiple definition	2 years ago	
.editorconfig	first nice commit	3 years ago	
.gitignore	修复 get_duration 结果不对问题	9 months ago	
CMakeLists.txt	update doc	5 months ago	
LICENSE	Initial commit	3 years ago	
MANIFEST.in	分割 pilk module	2 years ago	
README.md	update README.md	9 months ago	
pyproject.toml	first nice commit	3 years ago	
setup.py	update doc	5 months ago	

About

python silk codec bindings 支持微信语音编解码

pythonvoicemediacodecencodewechattencentpilkqq

ReadmeGPL-3.0 licenseActivityCustom properties55 stars0 watching11 forksReport repository

Releases 4

v0.2.4 Lateston May 26, 2023

+ 3 releases

Packages

No packages published

Languages



README GPL-3.0 license

pilk

python silk codec binding 支持微信语音编解码

pilk: python + silk

关联项目: [weixin-wxposed-silk-voice](#)

安装

python 3.6 | 3.7 | 3.8 | 3.9 | 3.10 | 3.11 Downloads 28k

pip install pilk

介绍与说明

[SILK](#) 是一种语音编码格式，由 [Skype](#) 公司研发，网上可找到的最新版本是 2012 发布的。

SILK 原始代码已上传到 [Release](#) , 包含规范文档

Tencent 系语音支持来自 [silk-v3-decoder](#)

[Release](#) 中也包含 [silk-v3-decoder](#) 重编译的 x64-win 版本，支持中文，[源代码](#)

SILK 编码格式 和 Tencent 系语音的关系：

此处 Tencent 系语音，仅以微信语音为例

- 标准 SILK 文件以 `b'#!SILK_V3'` 开始，以 `b'\xFF\xFF'` 结束，中间为语音数据
- 微信语音文件在标准 SILK 文件的开头插入了 `b'\x02'`，去除了结尾的 `b'\xFF\xFF'`，中间不变

已下统称为语音文件

语音数据

语音数据分为很多个独立 frame，每个 frame 开头两字节存储剩余 frame 数据的大小，每个 frame 默认存储 20ms 的音频数据

据此可写出获取 语音文件 持续时间(duration) 的函数（此函数 pilk 中已包含）

```
def get_duration(silk_path: str, frame_ms: int = 20) -> int:
    """获取 silk 文件持续时间，单位：ms"""
```

```
with open(silk_path, 'rb') as silk:
    tencent = False
    if silk.read(1) == b'\x02':
        tencent = True
    silk.seek(0)
    if tencent:
        silk.seek(10)
    else:
        silk.seek(9)
    i = 0
    while True:
        size = silk.read(2)
        if len(size) != 2:
            break
        i += 1
        size = size[0] + size[1] * 16
        silk.seek(silk.tell() + size)
    return i * frame_ms
```

根据 SILK 格式规范, `frame_ms` 可为 20, 40, 60, 80, 100

快速入门

详情请在 IDE 中查看 API 文档注释

在使用 `pilk` 之前, 你还需清楚 音频文件 `mp3`, `aac`, `m4a`, `flac`, `wav`, ... 与 语音文件 之间的转换是借助 [PCM raw data](#) 完成的

具体转换关系: 音频文件 $\leftrightarrow$ PCM $\leftrightarrow$ 语音文件

1. 音(视)频文件 $\rightarrow$ PCM

借助 `ffmpeg`, 你当然需要先用 [ffmpeg](#)

```
ffmpeg -y -i <音(视)频输入文件> -vn -ar <采样率> -ac 1 -f s16le <PCM输出文件>
```

- `-y`: 可加可不加, 表示 `<PCM输出文件>` 已存在时不询问, 直接覆盖
- `-i`: 没啥好说的, 固定的, 后接 `<音(视)频输入文件>`
- `-vn`: 表示不处理视频数据, 建议添加, 虽然不加也不会处理视频数据(视频数据不存在转PCM的说法), 但可能会打印警告
- `-ar`: 设置采样率, 可选的值是 [8000, 12000, 16000, 24000, 32000, 44100, 48000], 这里你可以直接理解为声音质量
- `-ac`: 设置声道数, 在这里必须为 1, 这是由 `SILK` 决定的
- `-f`: 表示强制转换为指定的格式, 一般来说必须为 `s16le`, 表示 16-bit short integer Little-Endian data
- example1: `ffmpeg -y -i mv.mp4 -vn -ar 44100 -ac 1 -f s16le mv.pcm`
- example2: `ffmpeg -y -i music.mp3 -ar 44100 -ac 1 -f s16le music.pcm`

2. PCM $\rightarrow$ 音频文件

```
ffmpeg -y -f s16le -i <PCM输入文件> -ar <采样率> -ac <声道数> <音频输出文件>
```

- `-f`: 这里必须为 `s16le`, 同样也是由 `SILK` 决定的
- `-ar`: 同上
- `-ac`: 含义同上, 值随意
- `<音频输出文件>`: 扩展名要准确, 没有指定格式时, `ffmpeg` 会根据给定的输出文件扩展名来判断需要输出的格式
- example3: `ffmpeg -y -f s16le -i test.pcm test.mp3`

`ffmpeg` 也可以使用 `python ffmpeg binding` 替换, 推荐 [PyAV](#) 大家自行研究, 这里不再啰嗦。

讲完了 音频文件 $\leftrightarrow$ PCM, 接下来就是用 `pilk` 进行 PCM $\leftrightarrow$ 语音文件 互转

silk 编码

```
import pilk

# pcm_rate 参数必须和 使用 ffmpeg 转 音频 到 PCM 文件时, 使用的 `~ar` 参数一致
# pcm_rate 参数必须和 使用 ffmpeg 转 音频 到 PCM 文件时, 使用的 `~ar` 参数一致
# pcm_rate 参数必须和 使用 ffmpeg 转 音频 到 PCM 文件时, 使用的 `~ar` 参数一致
duration = pilk.encode("test.pcm", "test.silk", pcm_rate=44100, tencent=True)

print("语音时间为:", duration)
```

silk 解码

```
import pilk

# pcm_rate 参数必须和 使用 ffmpeg 转 音频 到 PCM 文件时, 使用的 `~ar` 参数一致
```

```
duration = pilk.decode("test.silk", "test.pcm")

print("语音时间为:", duration)
```

使用 Python 转任意媒体文件到 SILK

使用 [pudub](#) 依赖 [ffmpeg](#)

```
import os, pilk
from pydub import AudioSegment

def convert_to_silk(media_path: str) -> str:
    """将输入的媒体文件转出为 silk, 并返回silk路径"""
    media = AudioSegment.from_file(media_path)
    pcm_path = os.path.basename(media_path)
    pcm_path = os.path.splitext(pcm_path)[0]
    silk_path = pcm_path + '.silk'
    pcm_path += '.pcm'
    media.export(pcm_path, 's16le', parameters=['-ar', str(media.frame_rate), '-ac', '1']).close()
    pilk.encode(pcm_path, silk_path, pcm_rate=media.frame_rate, tencent=True)
    return silk_path
```

使用 [pyav](#) 推荐

```
import os

import av

import pilk

def to_pcm(in_path: str) -> tuple[str, int]:
    """任意媒体文件转 pcm"""
    out_path = os.path.splitext(in_path)[0] + '.pcm'
    with av.open(in_path) as in_container:
        in_stream = in_container.streams.audio[0]
        sample_rate = in_stream.codec_context.sample_rate
        with av.open(out_path, 'w', 's16le') as out_container:
            out_stream = out_container.add_stream(
                'pcm_s16le',
                rate=sample_rate,
                layout='mono'
            )
            try:
                for frame in in_container.decode(in_stream):
                    frame.pts = None
                    for packet in out_stream.encode(frame):
                        out_container.mux(packet)
            except:
                pass
    return out_path, sample_rate

def convert_to_silk(media_path: str) -> str:
    """任意媒体文件转 silk, 返回silk路径"""
    pcm_path, sample_rate = to_pcm(media_path)
    silk_path = os.path.splitext(pcm_path)[0] + '.silk'
    pilk.encode(pcm_path, silk_path, pcm_rate=sample_rate, tencent=True)
    os.remove(pcm_path)
    return silk_path
```

